SW개발/HW제작 설계서

프로젝트 명 : **SLAM과 LiDAR를 이용한 구내 통신 로봇**

2025.10.30

(팀명) 행복회로

수행 단계별 주요 산출물

단계	산출물	일반 ·모바일 APP ·Web 등	응용 소프트웨어 ·빅데이터 ·인공지능 ·블록체인 등	응용 하드웨어 ·IoT ·로봇 ·드론 등
환경 분석	시장/기술 환경 분석서	△	△	△
	설문조사 결과서	△	△	△
	인터뷰 결과서	△	△	△
요구사항 분석	요구사항 정의서	○	○	○
	유즈케이스 정의서	△	△	△
아키텍처 설계	서비스 구성도(시스템 구성도)	○	○	○
	서비스 흐름도(데이터 흐름도)	△	○	△
	UI/UX 정의서	△	△	△
	하드웨어/센서 구성도	-	-	○
기능 설계	메뉴 구성도	○	○	○
	화면 설계서	○	○	△
	엔티티 관계도	○	○	△
	기능 처리도(기능 흐름도)	○	○	○
	알고리즘 명세서/설명서	△	○	○
	데이터 수집처리 정의서	-	○	-
	하드웨어 설계도	-	-	○
개발 / 구현	프로그램 목록	○	○	○
	테이블 정의서	○	○	△
	핵심 소스코드	○	○	○

※ ○ 필수, △ 선택

| 시장/기술 동향 분석

경향신문 • A10면 1단 • 2024.05.08. • 네이버뉴스

‘중대재해 최다’ DL이앤씨에서 또 사망**사고**..벌써 9번째 사망자

2022년 3월13일 DL이앤씨가 시공하는 서울 종로구 **건설현장**에서 작업자가 **전선** 포설 작업을 하다가 이탈된 드럼에 맞아 숨졌다. 같은 해 4월6일 경기 과천에서 굴착기와 기둥 사이 끼임 **사고**로 1명이 숨졌고, 8월5일 ...

덕트는 좁고 높아 사람이 접근하기 어렵고 위험한 환경으로 안전사고에 취약하기에 이를 개선하기 위한 시스템의 구축이 필요하다.

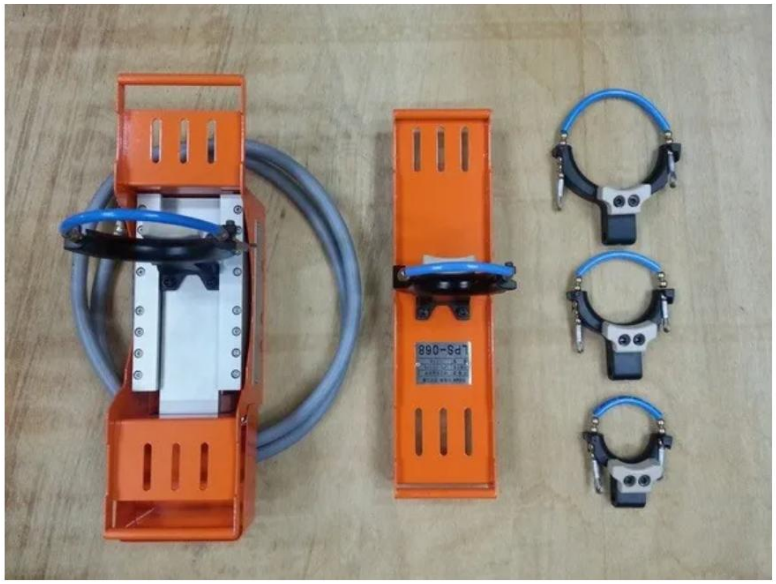
경제 기업PR

대우조선해양 업계 최초로 '전선 포설 로봇' 개발

안전성 높은 공압 방식 포설장치로 전선 포설의 신 기원 열어

인건비 150억 등 선박 건조 비용 절감 및 건조기간 단축...근골격계 질환 예방 기대

수정 2019-10-19 11:24 등록 2014-06-30 11:11



자료 제공 : 대우조선해양-전선 포설 로봇의 현장 적용 모습

케이블 덕트

케이블 트레이

출처 : 한국전기자재

작업공간에서 로봇 사용의 이점

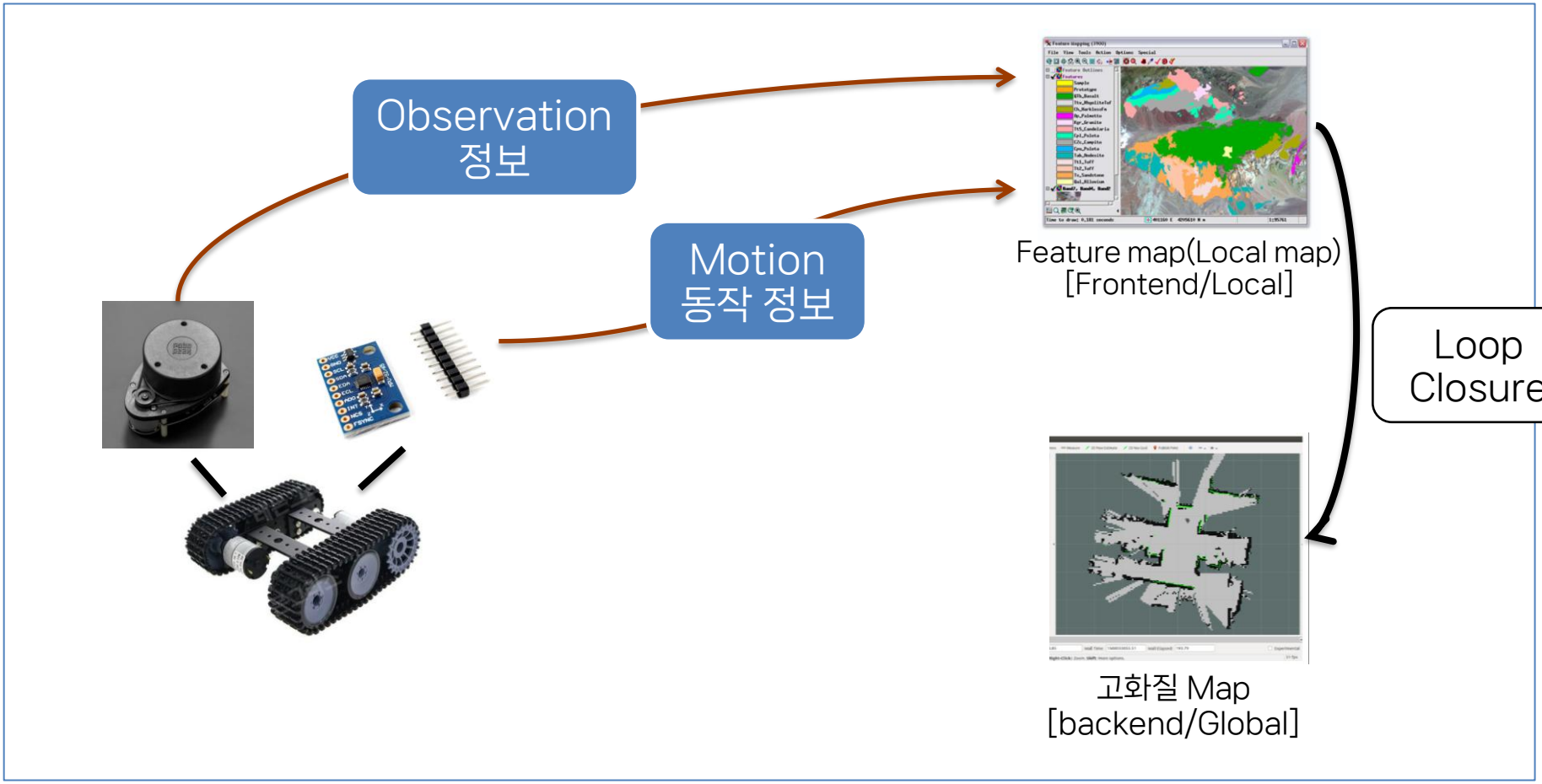
- 안전성 향상 : 높은 곳이나 좁은 공간 등 위험한 환경에서 작업을 할 필요가 줄어든다.
- 작업 시간 단축 : 로봇은 일정한 속도로 일을 수행하며, 작업의 일관성과 효율성을 높여 준다.
- 인력 비용 절감 : 반자동화된 로봇 시스템을 도입하면 작업 인력의 필요성이 줄어든다.
- 정밀성 향상 : 사람이 하기 힘든 복잡하고 정교한 작업이 가능하다.

| 요구사항 정의서

구분	기능	설명
S/W	위치정보 추적	IoT를 활용하여 위치 및 환경 데이터를 실시간 수집하고, 오픈소스 H/W 및 S/W와 연동하여 로봇의 위치정보를 제공하는 기능..
	AI 데이터분석 및 케이블 손상체크	덕트 내부 환경을 실시간으로 분석하고, 이상 징후(과부하, 온도 상승, 케이블 손상체크 등)를 감지하는 AI 기반 데이터 분석 기능.
	SLAM	Simultaneous Localization and Mapping 기술을 적용하여, 덕트 내부에서 로봇이 실시간으로 위치를 추적하고 지도를 작성하는 기능.
	비상 대응 시스템	화재, 단락, 전류 이상 감지 또는 예상치 못한 사고 발생 시 즉각적인 알람 및 대응이 가능하도록 IoT 기반 원격 모니터링 시스템과 연계.
	원격제어 및 진단	로봇의 실시간 제어 및 상태 진단이 가능하도록 클라우드 및 네트워크 기술 기반 원격 유지보수 기능 제공.
	자율주행	SLAM과 LiDAR 센서를 활용해 덕트 내부를 실시간으로 정밀하게 매핑하고, 로봇의 위치를 정확히 추정하며 자율적으로 주행. 장애물 회피 및 최적 경로 탐색 기능을 통해 복잡하고 협소한 환경에서도 안정적인 이동과 점검을 수행

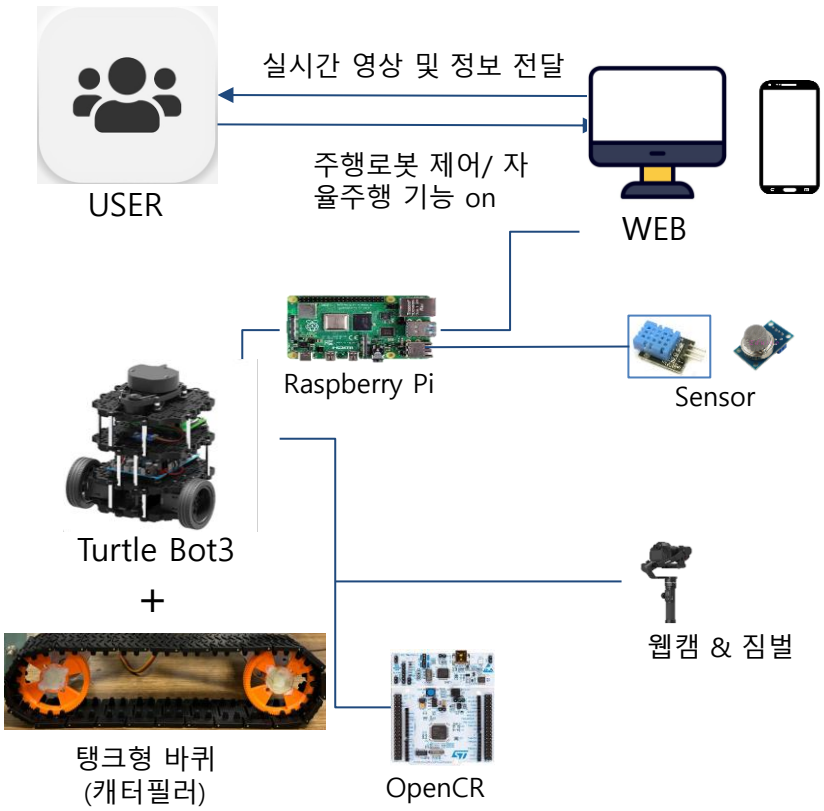
구분	기능	설명
H/W	덕트 모델환경	상업 및 산업용 건물 내 덕트 구조를 모델링하고, 다양한 환경에서 설계한 덕트 관리 로봇을 테스트할 수 있도록 모의 환경을 구축.
	이동체 / 로봇	덕트의 내부 크기에 상관없이 원활히 이동할 수 있는 자율주행 기반 이동체(로봇) 개발. 케이블 포설 및 설치 기능도 지원 가능.
	배터리 부족 대응 시스템	로봇이 덕트 내부에서 사람의 개입없이 장시간 작동할 수 있도록 무선 충전 기술을 적용하여 지속적인 운용 가능. 또한 덕트 내부에 충전소가 설치된 경우, 로봇 배터리 부족 시에 SLAM 기반 내비게이션을 통해 충전소로 이동하도록 설계

| SLAM 서비스 구성도 - 서비스 시나리오



참고문헌) W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2D LIDAR SLAM," Google.

| 서비스 구성도 - 서비스 시나리오



덕트 주행

1. 사용자가 웹페이지로 로봇 제어/자율주행 기능 ON
2. 실시간 안정적인 주행 영상 확인 가능 (Raspberry PI 카메라 활용)
3. TurtleBot3 플랫폼에 탱크형 바퀴를 결합하여 접지력을 높이고 덕트의 폭보다 작게 설계하여 보다 안정적인 주행 가능

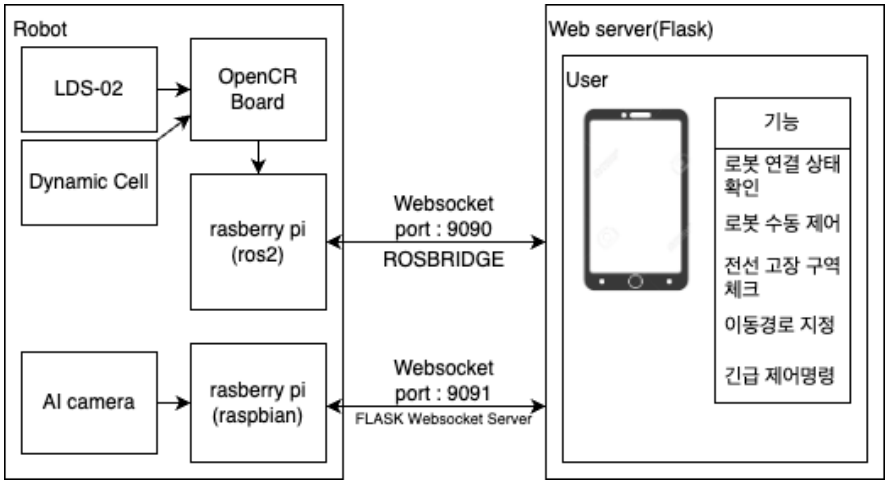
SLAM Mapping 및 자율 주행

1. LiDAR 센서를 활용하여 Cartographer SLAM 기술을 활용해 3D 지도 생성
2. 목적지를 향해 자율주행

웹 기반 실시간 모니터링 및 현장 환경 점검 자동화

1. 사용자가 웹페이지로 로봇의 배터리 상태 및 상태 정보를 확인 가능
2. 케이블 손상 및 이상 환경 감지 가능 & AtlasDB를 통해 손상 데이터 저장 및 관리 가능

| 서비스 흐름도



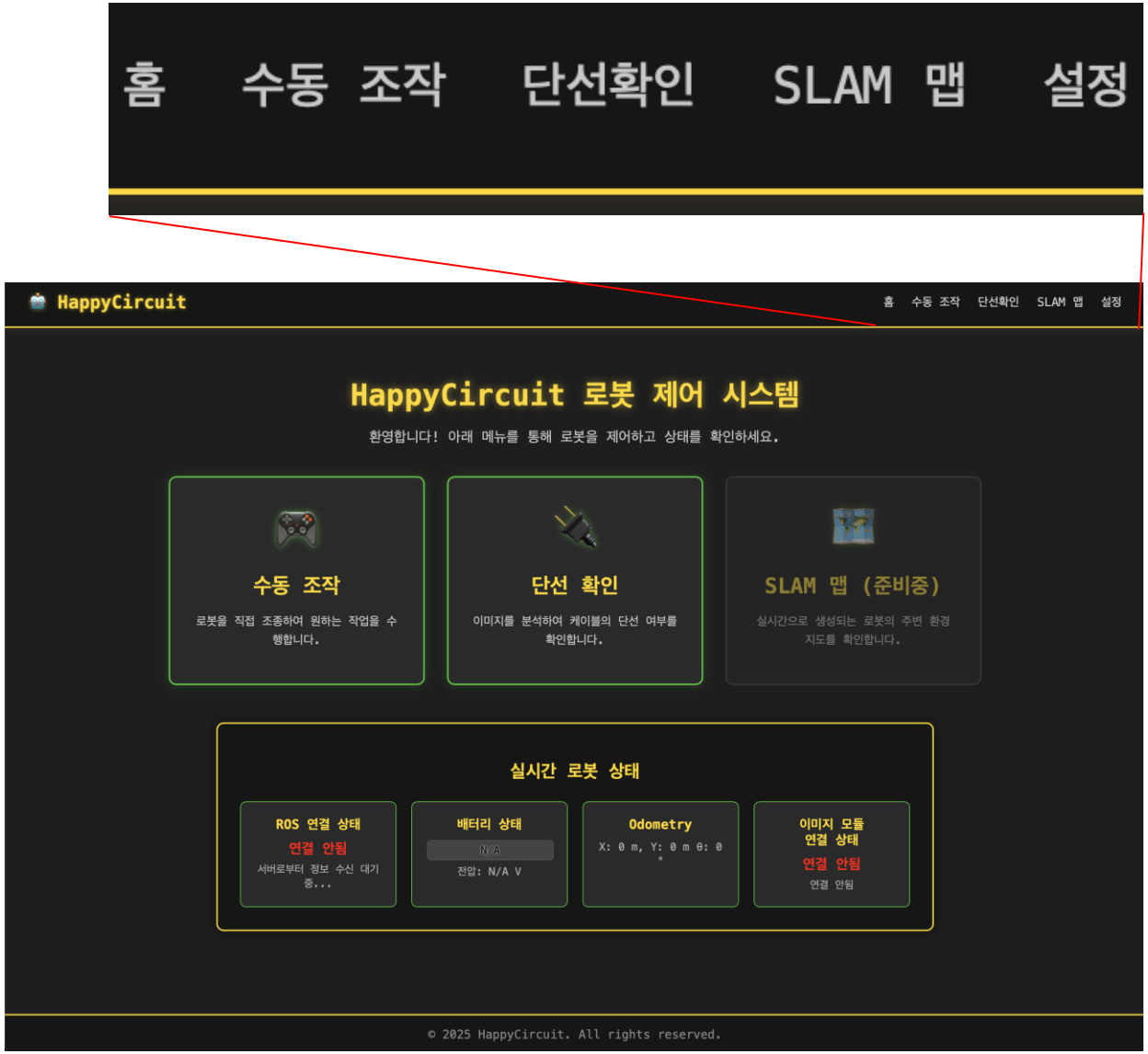
○ **User(사용자 앱, 웹)**

- 안드로이드 앱에서(혹은 웹에서) 로봇 상태를 확인하고, 로봇 제어 명령을 서버에 요청한다.
- 사용자가 앱을 통해 로봇의 목적지 및 필요시 이동경로를 지정할 수 있고, 전선 포설 및 철거 작업 명령, 비상 상황시 긴급 제어 명령을 내린다.

○ **HW(로봇)**

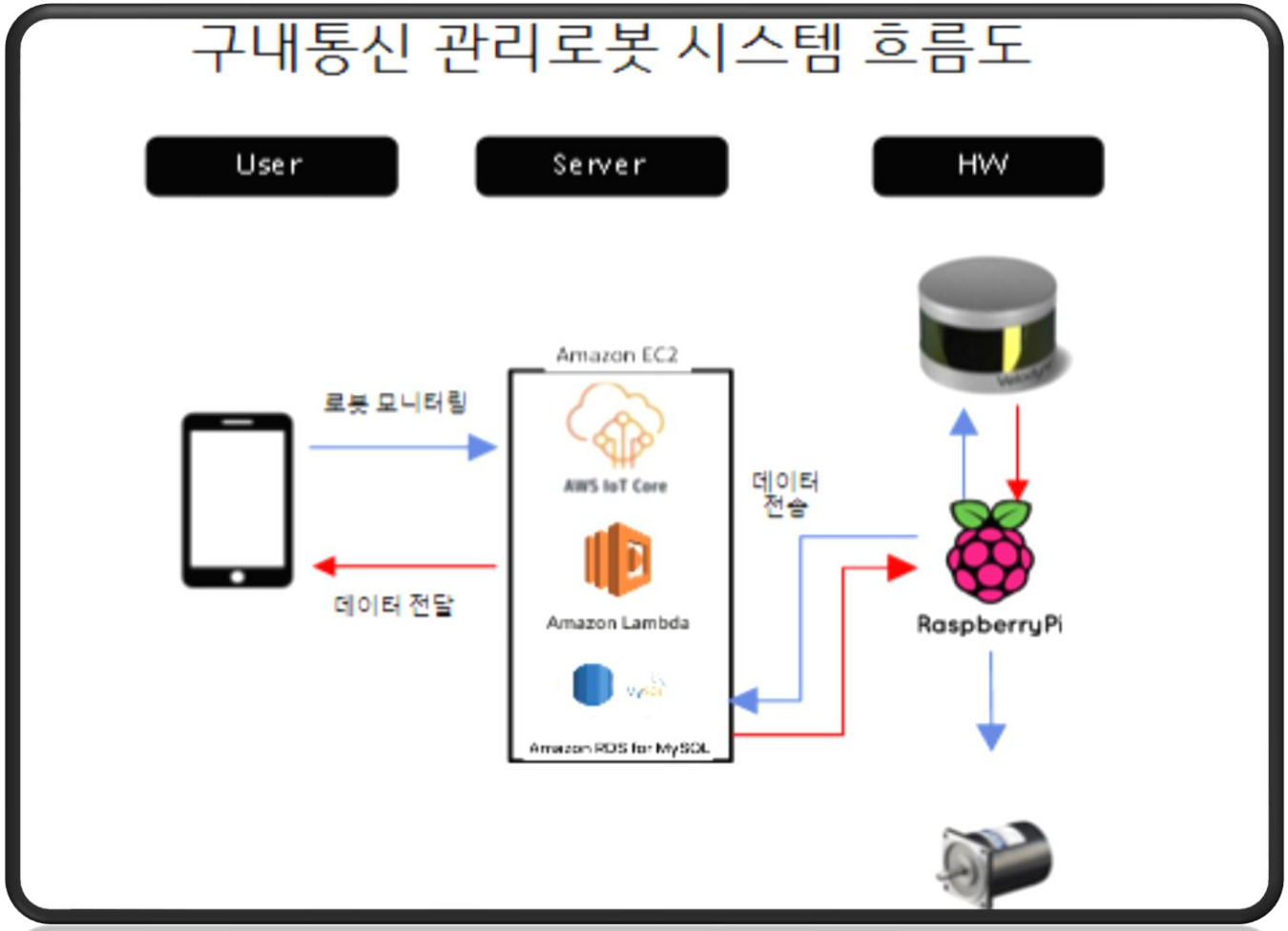
해당 영역에서는 라즈베리 파이를 이용해 지능제어를 구현함으로써 로봇이 이용 목적대로 움직이도록 합니다. 라즈베리 파이와 오픈소스 H/W 를 이용하여 로봇에 구현하고자 하는 지능제어를 구현합니다.

| UI/UX 정의서 - 화면 설계서



1	**홈** ▪ 로봇의 CV모듈과 SLAM 모듈 연결상태 확인 가능 ▪ 로봇의 배터리 상태 및 로봇의 현재 위치와 방향 표시
1	**수동조작** ▪ 사용자가 직접 로봇 주행을 원할 경우 해당 기능 사용 ▪ **웹페이지에서의 버튼을 클릭하여 사용자가 직접 조종**
2	**SLAM 맵** ▪ 로봇이 주행 중 mapping 한 지도 확인 가능
3	**설정** ▪ 화면 URL 및 글씨, 글자 수동 설정 가능 ▪ 홈페이지의 기본 설정 조작

| 기능 처리도(기능 흐름도)


구내통신 관리로봇 시스템 흐름도
User
Server
HW
Amazon EC2
로봇 모니터링
데이터 전달
AWS IoT Core
Amazon Lambda
Amazon RDS for MySQL
데이터 전송
RaspberryPi

YOLO 기반 케이블 손상 객체 탐지 알고리즘

YOLO(You Only Look Once)는 객체 탐지(Object Detection)를 위한 **단일 단계**(one-stage) 딥러닝 모델로, 이미지를 한 번만 보는 단일 전방향 신경망 추론으로 여러 객체의 위치(bounding box)와 종류(class)를 동시에 예측하는 알고리즘이다. 이는 기존의 **R-CNN** 계열처럼 영역 후보(region proposal)를 여러 번 분류하는 **두 단계**(two-stage) 방식과 달리, **하나의 통합된 신경망**이 전체 입력 이미지에서 **한 번에** 객체들의 경계상자와 클래스 확률을 출력한다.

YOLO 기반 케이블 손상 객체 탐지 알고리즘

YOLO(You Only Look Once)는 객체 탐지(Object Detection)를 위한 **단일 단계**(one-stage) 딥러닝 모델로, 이미지를 한 번만 보는 단일 전방향 신경망 추론으로 여러 객체의 위치(bounding box)와 종류(class)를 동시에 예측하는 알고리즘이다. 이는 기존의 **R-CNN** 계열처럼 영역 후보(region proposal)를 여러 번 분류하는 **두 단계**(two-stage) 방식과 달리, **하나의 통합된 신경망**이 전체 입력 이미지에서 **한 번에** 객체들의 경계상자와 클래스 확률을 출력한다.

| 알고리즘 상세 설명서

Explorer 자율 탐색 알고리즘 (Frontier-based Exploration)

알고리즘은 지도 데이터를 분석하여, 이미 '인지된 공간(Known Space)'과 '미지의 공간(Unknown Space)'이 맞닿아 있는 경계 영역(Frontier)을 수학적으로 식별하고 추출한다.

추출된 다수의 경계 영역 중, 사전에 정의된 비용 함수(예: 현재 로봇 위치에서 가장 가까운 경계 지점)에 따라 최적의 탐색 지점을 동적으로 선정한다.

선정된 경계 지점의 좌표는 ROS2 내비게이션 스택(Nav2)의 새로운 목표 지점으로 발행된다.

Nav2는 이 목적지까지 장애물을 회피하며 로봇을 주행시키고, 로봇이 이동함에 따라 SLAM(Cartographer)은 맵을 갱신하며 미지의 영역을 확장해 나간다.

explorer.py

▼ source

```
class ExplorerNode(Node):
    def __init__(self):
        super().__init__('explorer')
        self.get_logger().info("Explorer Node Started")

        # --- \uc0c1\ud0dc \uad00\ub9ac \ubcc0\uc218 ---
        # 'EXPLORING': \ud0d0\uc0c9 \uc911
        # 'RETURNING_HOME': \uc2dc\uc791 \uc9c0\uc810\uc73c\ub85c \ubcf5\uadc0 \uc911
        # 'SHUTTING_DOWN': \uc885\ub8cc \uc911
        self.state = 'EXPLORING'
        self.get_logger().info(f"Initial Setting. state : EXPLORING")
        self.start_position = None # \ud0d0\uc0c9 \uc2dc\uc791 \uc704\uce58\ub97c \uc
        self.frontier_failure_count = 0

        # Subscriber to the map topic
        self.map_sub = self.create_subscription(
            OccupancyGrid, '/map', self.map_callback, 10)
```

- Explorer(자율탐색 알고리즘) 코드 일부

| 핵심 소스코드(1)

- Web Server Code

- Flask & SocketIO Setup

```
# --- Flask 및 SocketIO 앱 초기화 ---
app = Flask(__name__)
# --- Flask 앱에 /control 루트 추가 ---
app.register_blueprint(control_bp)
app.register_blueprint(disconnection_check_bp)
```

- ROSBridge Client Thread

```
self.ros_client = roslibpy.Ros(host=self.ros_host, port=self.ros_port)
odom_listener.subscribe(self.odom_callback)
self.robot_controller = SmoothRobotController(self.cmd_vel_publisher)
self.robot_controller.start()
```

- Image Client Thread

```
class ImageClientThread(threading.Thread):
    def __init__(self, socketio_instance):
        super().__init__()
        self.daemon = True
        self.socketio = socketio_instance
        self.is_running = True
        self.ws = None
        self.host = config.PI_CV_WEBSOCKET_HOST
        self.port = config.PI_CV_WEBSOCKET_PORT
```

| 핵심 소스코드(2)

- 라즈베리파이 flask 서버 코드 (실시간 스트리밍)

- 카메라 설정 및 영상 스트리밍

```
app = Flask(__name__)
capture = cv2.VideoCapture(-1)
capture.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

- 영상프레임 생성 함수

```
def gen_frames():
    while True:
        success, frame = capture.read()
        if not success:
            break
        else:
            ret, buffer = cv2.imencode('.jpg', frame)
            frame = buffer.tobytes()
            yield (b'--frame\r\n'
                   b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')
```

- gen_frames() 함수에서 생성된 프레임을 반환

```
@app.route('/video_feed')
def video_feed():
    return Response(gen_frames(), mimetype='multipart/x-mixed-replace; boundary=fr
```

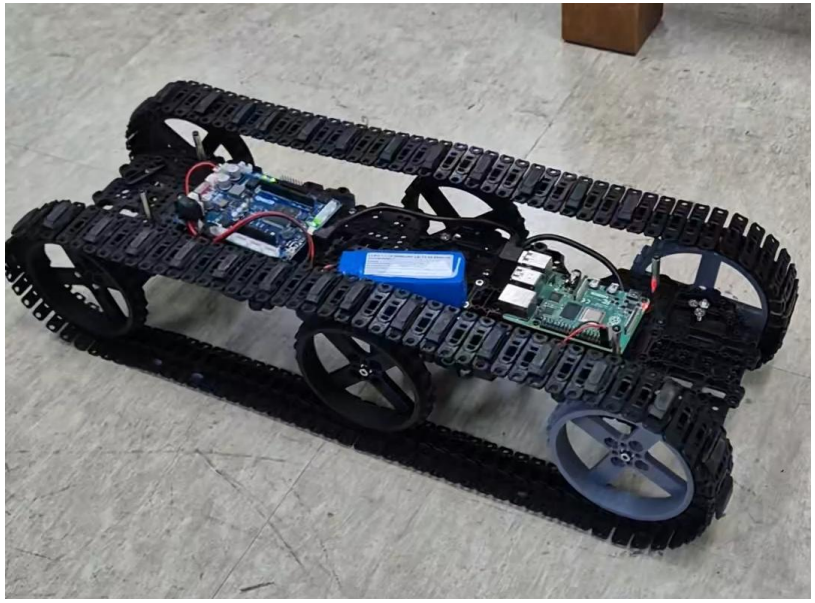
| 참조-개발 환경 및 설명

구분	항목	적용내역
S/W 개발환경	OS	Ubuntu Linux, ROS2
	개발환경(IDE)	Intellij, MobaXterm, ROS2 개발 패키지
	개발도구	ROS2, Nav2, cartographer, Git, AWS IoT Core, Colab
	개발언어	Python, C++, Bash Script
	기타사항	AWS Lambda, 오픈소스 라이브러리
H/W 구성장비	디바이스	라즈베리파이4, TurtleBot3 burger
	센서	LDS-02 LiDAR, RPLIDAR A3, Pi Camera, 온습도 센서(DHT22), LED센서
	통신	Wi-Fi, LTE/5G, DDS, AWS IoT Core
	언어	Python, C++, ROS2 메시지 언어
	기타사항	배터리팩, 각종 모터 및 바퀴, IMU 센서 등

| 참조-H/W 기능 실사 사진

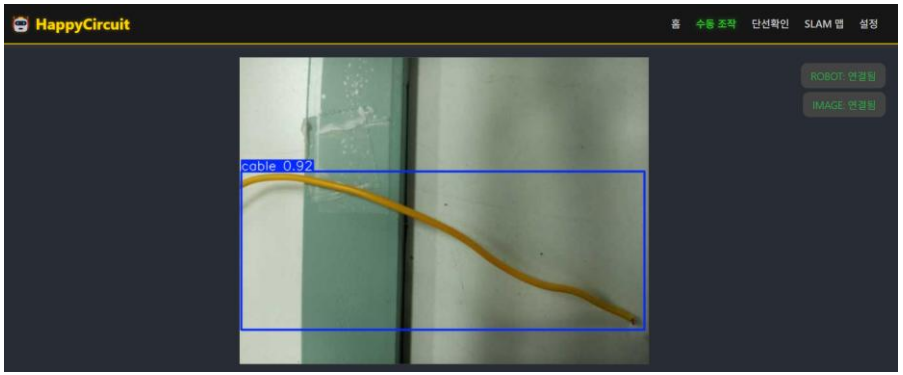
로커-보기 형태 로봇 + 탱크형바퀴

장애물을 보다 유연하게 극복하고 덕트사이에 발이 빠지는 문제와 접지력 향상을 위해 터틀봇 로봇과 탱크형 바퀴를 결합

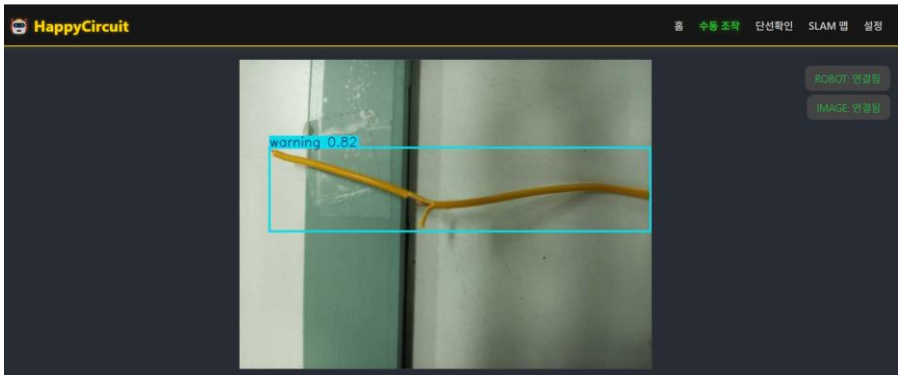


| 참조-S/W 기능 실사 사진

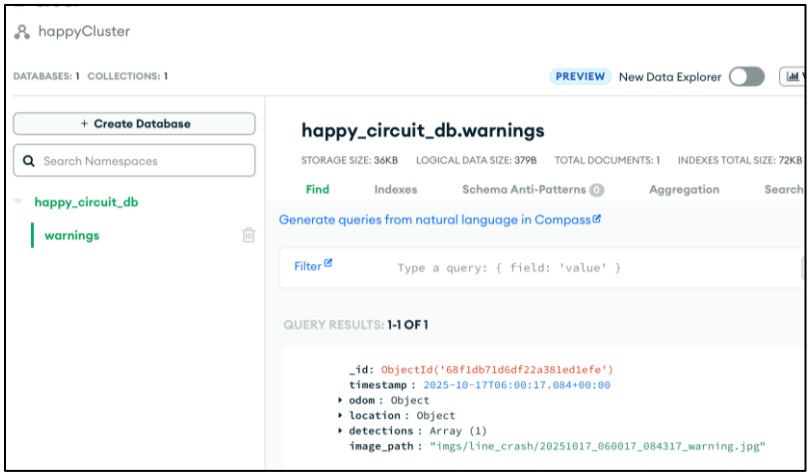
웹 기반 실시간 모니터링 시스템 및 DB 구축



(정상 케이블 객체 탐지)



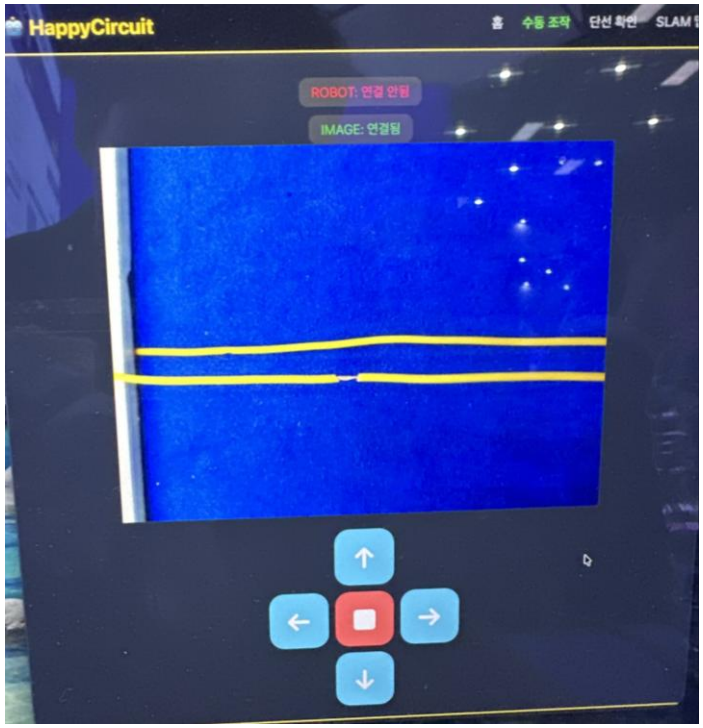
(손상 케이블 객체 탐지)



(Atlas DB에 저장된 손상 케이블 데이터)

| 참조-S/W 기능 실사 사진

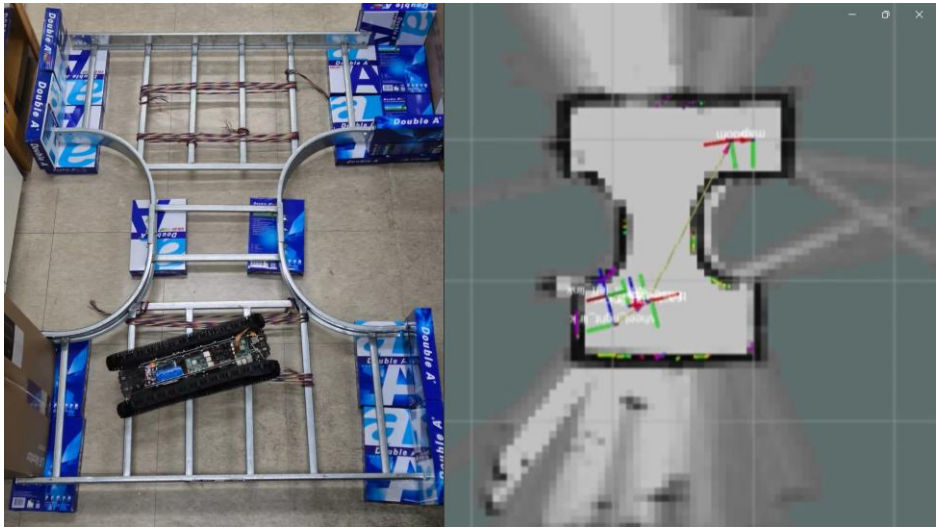
웹 기반 실시간 수동조작



(웹 내의 수동조작 메뉴 화면)

- 웹 인터페이스의 '수동 조작' 탭에서 방향키 입력을 통해 로봇의 수동 제어가 가능하다.

SLAM Mapping 및 자율주행



(Cartographer SLAM을 이용한 덕트 환경 내 실시간 지도 작성 결과)

- 덕트 내부의 제한된 공간에서도 로봇은 뛰어난 주행 안정성을 보이며, 주행과 동시에 주변 환경을 Mapping하는 기능도 원활하게 수행한다.

| 참조-프로젝트 관리

프로젝트 관리

18시간 전 편집

OpenCV Rasberrypi websocket steam server open

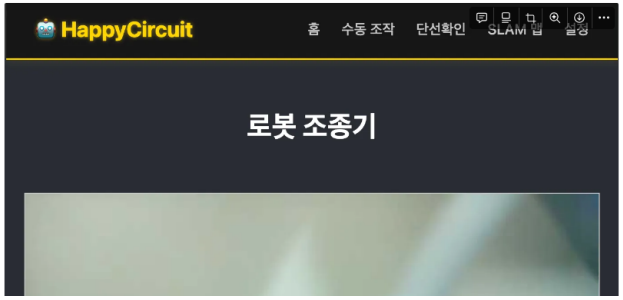
- websocket 서버 설정을 위해 "happy_websocket" 가상환경 구축

```
source happy_websocket/bin/activate
```

해당 명령어를 shell 에 입력해 가상환경에 접속

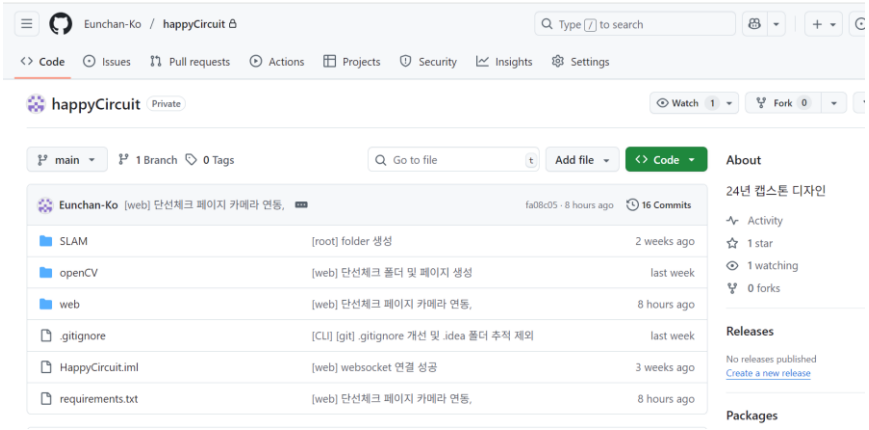
접속 후 하단의 명령어를 이용해 서버 실행

```
python pi_stream_ws_server.py
```



- Notion을 통해 매주 혹은 매달 프로젝트 진행사항 기록

형상관리



- Github를 통해 개발 코드 및 소스 파일 공유

Thank you